

[L] 李硕 · 前端开发工作汇报 [R] 转转 [C]1

前端开发技术报告

校园二手物品交易平台——转转

李硕 · 前端开发

华中农业大学 · 2026 年 7 月

目录

1	个人角色与工作概述	3
2	技术栈说明	3
3	首页开发	3
3.1	Hero 横幅	3
3.2	公告栏滚动轮播	4
3.3	商品分类导航	4
3.4	商品网格	4
4	商品模块	4
4.1	商品列表页	4
4.2	商品详情页	5
4.3	商品发布页	5
5	购物车模块	5
6	订单列表页	6
7	管理后台	6
7.1	AdminLayout 布局	6
7.2	AdminUsers 用户管理页	6
7.3	AdminProducts 商品审核页	7
7.4	AdminOrders 订单管理页	7
7.5	AdminAnnouncements 公告管理页	7
8	ECharts 数据可视化	7
8.1	分类占比饼图	7
8.2	每日趋势折线图	7
9	种子数据与图片方案	8
9.1	155 件商品数据编写	8
9.2	自定义产品图片生成方案	8
10	代码贡献统计	8
11	核心技术亮点	8
12	商品列表筛选系统设计	9

目录	2
13 管理后台系统架构	10
14 ECharts 图表实现详解	10
14.1 饼图实现	10
14.2 折线图实现	11
14.3 内存泄漏防护	11
15 购物车状态管理	11
16 商品发布页的图片上传	11
17 种子数据与图片方案的迭代过程	12
18 响应式设计实现	12
19 项目中的技术决策	13
20 遇到的问题与解决方案	13

1 概述

我在项目中负责前端页面的开发和数据准备工作。具体来说：

- 首页（Hero 横幅、商品网格、统计分析）
- 商品模块（列表搜索筛选、详情展示、发布表单）
- 购物车（选品、改量、结算）
- 订单列表（买家卖家双视角切换）
- 管理后台（5 个管理页面 + 侧边栏布局）
- ECharts 数据可视化图表
- 155 件商品种子数据和图片生成

以下按模块逐一说明做了什么、怎么做的、为什么这么做。

2 首页是怎么工作的

2.1 数据加载流程

当你打开首页（/），页面同时发出 4 个请求：

- GET /api/v1/index/statistics → 获取统计数据（用户数、商品数、订单数）
- GET /api/v1/category/list → 获取分类列表（6 个一级 + 20 个二级）
- GET /api/v1/announcement/list → 获取公告列表
- GET /api/v1/product/list?page=1&size=20 → 获取首页商品

这四个请求是并行发出的（不是等一个完成再发下一个），这样可以更快地加载页面。每个请求都有独立的 try-catch，一个失败了不影响其他。

2.2 商品网格的响应式布局

商品卡片排列使用 CSS Grid 布局：

```
.product-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(240px, 1fr));  
  gap: 20px;  
}
```

通俗解释：让浏览器自动计算一行能放下几个卡片。每个卡片最少 240px 宽，如果屏幕够宽就多放几个，屏幕窄就少放几个。auto-fill 表示自动填充，1fr 表示每个卡片平均分配剩余空间。

2.3 时间格式化

商品列表中的时间是绝对时间（如“2026-07-05 14:30:00”），但展示给用户时应该用相对时间。实现逻辑：

```
function formatTime(t) {
  const d = new Date(t)           // 解析时间字符串
  const now = new Date()          // 当前时间
  const diff = now - d             // 时间差（毫秒）
  if (diff < 60000) return '刚刚' // < 1分钟
  if (diff < 3600000) return Math.floor(diff/60000) + '分钟前' //
    < 1小时
  if (diff < 86400000) return Math.floor(diff/3600000) + '小时前'
    // < 1天
  return d.toLocaleDateString('zh-CN') // 超过1天显示日期
}
```

2.4 公告栏滚动

公告栏水平滚动用纯 CSS 实现，不需要 JavaScript 定时器。

核心原理：复制一份公告内容放在末尾，然后让整个内容从右到左无限移动。当第一份内容完全滚出屏幕时，它无缝衔接到末尾的副本继续滚动，视觉上形成无限循环。

```
@keyframes noticeScroll {
  0% { transform: translateX(0); }
  100% { transform: translateX(-50%); } /* 移动一半宽度 */
}
.notice-track {
  animation: noticeScroll 40s linear infinite; /* 40秒一圈，匀速，
    无限 */
}
.notice-board:hover .notice-track {
  animation-play-state: paused; /* 鼠标悬停暂停 */
}
```

3 商品列表页的筛选系统

3.1 筛选参数如何在 URL 中保持

商品列表页有 5 个筛选条件：关键词、分类、价格区间、成色、排序。如果用户筛选后刷新页面，条件不应丢失。

实现方法：把所有筛选参数写入 URL query string。例如：`/product/list?keyword=iPhone&cate`

```
// 当用户改变筛选条件时，更新 URL
function applyFilters() {
  const query = {}
  if (keyword.value) query.keyword = keyword.value
  if (categoryId.value) query.categoryId = categoryId.value
  // ...
  router.push({ query }) // 更新 URL，不刷新页面
}

// 页面初始化时，从 URL 读取筛选条件
onMounted(() => {
  if (route.query.keyword) keyword.value = route.query.keyword
  if (route.query.categoryId) categoryId.value = Number(route.query
    .categoryId)
  // ...
  loadProducts()
})
```

3.2 前端如何实现分页

分页组件（Element Plus的 `el-pagination`）维护两个关键数据：

- `page`：当前第几页
- `size`：每页显示多少条（默认 20）

每次翻页时，前端发送 GET `/api/v1/product/list?page=2&size=20`，后端返回该页的数据和总条数 `total`。前端用 `total` 除以 `size` 算出总页数，显示在分页组件上。

3.3 分类级联选择器

商品发布时需要选择分类（如“电子产品 > 手机”）。使用 Element Plus 的 `Cascader` 组件，数据来自 `/category/list` 接口，返回的 JSON 已经是嵌套结构（父级含 `children`

数组), Cascader 可以直接使用。

```
[
  { "id":1, "name":"电子产品", "children":[
    {"id":7, "name":"手机"},
    {"id":8, "name":"电脑"}
  ]},
  { "id":2, "name":"书籍教材", "children":[...]}
]
```

用户选择”手机”后, Cascader 返回的值是 [1, 7] (父级 ID + 子级 ID), 提交表单时取最后一个元素 7 作为 categoryId。

4 商品详情页

4.1 HTML 安全过滤

商品描述可能包含 HTML(富文本)。如果不加过滤,恶意用户可以注入 `<script>alert('攻击')</script>`, 这在其他用户浏览时会被执行 (XSS 攻击)。

过滤实现分三步:

1. `.replace(/<script[>]*>.*?<script>/gi,"")` — `<script>`

1. `.replace(/<style[>]*>.*?<style>/gi,"")` — `<style>`

1. `.replace(/<[>]*>/g,"")` — `HTML`

1. 通过 DOM 的 `textarea.innerHTML = ...; textarea.value` 解码 HTML 实体(如 `&` → `&`)

4.2 图片画廊

商品可以有多张图片 (product_image 表)。主图显示一张大图, 下方排列缩略图。点击缩略图切换主图, 通过改变 `currentImage` 响应式变量的值实现。

如果图片加载失败 (网络问题或 URL 失效), `<img>` 标签的 `@error` 事件会触发, 显示 Emoji 占位符 (如手机分类显示)。

5 购物车

5.1 全选/取消全选

全选状态通过计算属性动态判断:


```
// 当所有项的 selected 都为 true 时, 全选 = true
const allSelected = computed(() =>
  list.value.length > 0 && list.value.every(i => i.selected)
)

// 切换全选: 把所有项设为相同值
function toggleAll() {
  const newVal = !allSelected.value
  list.value.forEach(i => i.selected = newVal)
}
```

5.2 防抖 (Debounce)

数量调节按钮 (+/-) 如果用户快速连点, 会发送大量网络请求。防抖策略: 用户每次点击后不立即发送请求, 而是等 300ms, 如果 300ms 内又点击了, 就重新计时。只有最后一次点击后 300ms 内没有新点击, 才真正发送请求。

5.3 结算流程

点击”结算”→ 弹出地址选择对话框 → 用户选择或添加地址 → 确认下单 → 为每个选中商品调用 POST /api/v1/order → 跳转到支付页面。

地址选择对话框从 /api/v1/user/address 获取用户地址列表, 默认选中”默认地址”。

6 订单列表的双视角切换

这是订单列表最重要的功能改进。原始代码中 role 被写死为 'buyer', 卖家永远看不到自己的订单。

修复方案: 增加一个 El-Radio-Group 切换按钮——”我买的” (role=buyer) 和”我卖的” (role=seller)。切换时重新请求数据:

```
const role = ref('buyer') // 默认显示"我买的"

async function loadOrders() {
  const params = { role: role.value } // 动态切换角色
  if (activeStatus.value) params.status = activeStatus.value
  const res = await orderApi.getList(params)
  list.value = res.data || []
}
```

当切换到“我卖的”并选择“待发货”标签时，页面会显示绿色提示条，引导卖家进入订单详情发货。订单卡片上还有一个突出的“去发货”按钮，点击直接跳转。

7 管理后台

7.1 整体架构

管理后台有独立的布局（AdminLayout），使用侧边栏导航。路由配置中设置了 `meta: { requiresAdmin: true }`，路由守卫检查用户角色为非管理员时重定向到首页。

7.2 商品审核页

这是管理后台最复杂的页面。表格展示所有商品，筛选栏支持按状态（待审核/在售/已下架/审核驳回）过滤。操作按钮包括：

- 通过：调用 `PUT /api/v1/admin/product/review/{id}`，状态改为“在售”
- 驳回：同上，状态改为“审核驳回”
- 下架：调用 `PUT /api/v1/admin/product/{id}/offline`，状态改为“已下架”
- 编辑：弹出对话框，可修改标题、价格、成色、状态、描述

所有操作都有确认对话框（`ElMessageBox.confirm`），防止误操作。

筛选条件变化时自动重置页码到第一页：用 `watch` 监听筛选参数变化，在回调中设置 `page = 1`。

7.3 数据统计页——ECharts 图表

这是项目中唯一涉及数据可视化的页面，使用 ECharts 5 库。

图表初始化流程：

1. 在模板中放置一个 `<div ref="chartRef">` 作为图表的容器
2. 在 `onMounted` 中获取 DOM 引用，调用 `echarts.init(chartRef.value)` 创建图表实例
3. 调用 `chart.setOption({...})` 设置图表的配置和数据
4. 在 `onUnmounted` 中调用 `chart.dispose()` 销毁实例（释放内存）

饼图配置要点:

```
const option = {
  series: [{
    type: 'pie',
    radius: ['40%', '70%'],    // 环形图（内径40%，外径70%）
    data: [
      { name: '手机', value: 15 },
      { name: '电脑', value: 15 },
      // ...其他分类
    ],
    label: { show: true, formatter: '{b}: {c}件 ({d}%)' }
  ]
}
chart.setOption(option)
```

折线图配置要点:

```
const option = {
  xAxis: { type: 'category', data: ['7/1', '7/2', ...] }, // X轴 日期
  yAxis: [
    { type: 'value', name: '用户数' }, // 左Y轴
    { type: 'value', name: '订单数' } // 右Y轴
  ],
  series: [
    { name: '新注册用户', type: 'line', data: [...] },
    { name: '新订单', type: 'line', yAxisIndex: 1, data: [...] }
  ]
}
```

内存泄漏防护: ECharts实例持有 DOM 引用，组件卸载后如果不手动销毁，实例会一直占用内存。在 SPA 中频繁切换页面会积累大量未释放的图表实例。解决方案是在 `onUnmounted` 中调用 `chart.dispose()` 和 `removeEventListener`。

8 商品种子数据

8.1 数据是怎么生成的

155 件商品数据通过 SQL INSERT 语句批量写入。每件商品包含:

- 基本信息: 标题、描述、价格、原价
- 分类: categoryId (1-25, 对应 6 个一级 + 19 个二级分类)

- 成色：5 个枚举值之一
- 卖家：user_id 随机分配给 17 个用户
- 统计：viewCount、favoriteCount 随机赋值

8.2 图片是怎么生成的

经过 6 轮方案迭代，最终采用 Python Pillow 库自定义生成图片。脚本流程：

```
# 1. 从数据库读取所有商品
for pid, title, cat_id in products:
    # 2. 根据分类选择背景色
    color, label = cat_colors.get(cat_id)

    # 3. 创建 640x480 的图片
    img = Image.new('RGB', (640, 480), rgb)

    # 4. 在图片上绘制文字
    draw = ImageDraw.Draw(img)
    draw.text((320, 240), title, fill="white", anchor="mm")

    # 5. 保存为 JPG
    img.save(f"{pid}.jpg", quality=90)
```

每张图片都包含了该商品的分类标签和商品名称，确保图文 100% 对应。图片存储在 Docker 容器的 /data/zhuanzhuan/uploads/products/ 目录中，通过 Nginx 代理由后端提供访问。

为什么不用外部图库：在项目开发过程中，我们尝试了多种外部图片来源 (Lorem-Flickr、Picsum、Unsplash、Pexels)，但都存在问题——部分图片与商品内容不符、部分 URL 失效、部分服务不稳定。最终的本地生成方案虽然图片是简单的文字 + 色块，但它是 100% 可靠、100% 匹配、0 外部依赖的。

9 数据库：数据是怎么组织的

虽然不是直接写后端，但理解数据库结构对前端开发很重要。本项目的核心数据表：

- **user 表：**存储用户信息。关键字段：username (用户名)、password_hash (BCrypt 加密的密码)、role (user/admin)、status (0 禁用/1 启用)。有软删除 (deleted_at)。

- **product** 表：存储商品信息。关键字段：user_id (卖家)、category_id (分类)、title、price、status (待审核/在售/已下架/已售出/审核驳回)。有软删除和 FULLTEXT 全文索引。
- **order** 表：存储订单。关键字段：order_no (唯一订单号)、buyer_id、seller_id、product_id、status (待付款/待发货/待收货/已完成/已取消)。
- **cart** 表：购物车。联合唯一键 (user_id, product_id)，同一用户同一商品只保留一条。
- **message** 表：站内信。from_user_id (发送方)、to_user_id (接收方)、is_read (0 未读/1 已读)。
- **notification** 表：系统通知。user_id (接收方)、type (order/review/system)、is_read。

其中最重要的设计是 order 表的状态机——5 个状态之间有严格的转换规则：待付款 → 待发货 → 待收货 → 已完成，中间任何一步都可以转成“已取消”。每次状态变更都会 order_log 表中记录一条日志。

10 前后端如何协作

10.1 数据格式约定

前后端通过 JSON 格式通信。所有接口返回统一格式：

```
{
  "code": 200,           // 状态码：200 成功，4xx 客户端错误，5xx 服务端
                        错误
  "message": "success",
  "data": {...}         // 实际数据
}
```

前端 Axios 响应拦截器检查 code，如果是 200 就返回 data，否则弹出错误提示。

10.2 接口对接流程

以“发布商品”为例：

1. 前端收集表单数据（标题、分类、价格、描述、图片 ID 列表）
2. 前端通过 Axios 发送 POST /api/v1/product，请求头自动带 Token
3. 后端的 Spring Security 过滤器验证 Token

4. 验证通过后，请求到达 ProductController.publish() 方法
5. Controller 调用 ProductService.publish()，后者开启数据库事务
6. 事务中：插入 product 表 → 更新 product_image 表的 product_id → 设置封面图
7. 返回 JSON 结果给前端
8. 前端收到成功响应，跳转到商品详情页

10.3 错误处理策略

前端对网络错误做了分类处理：

- ECONNABORTED：请求超时（15 秒），提示” 检查网络连接”
- 无 response：网络断开，提示” 网络连接失败”
- 401：Token 过期，自动刷新 Token
- 403：权限不足。如果本地有 Token 说明用户可能被删除，清除登录
- 404：资源不存在
- 429：请求太频繁
- 5xx：服务器错误，提示” 稍后重试”

11 项目中的具体问题与解决

问题一：ECharts图表在窗口变大变小后不变。原因：ECharts 不会自动响应窗口尺寸变化。解决：监听 window resize 事件，回调中调用 chart.resize()。

问题二：管理后台筛选后跳到空页。原因：当前在第 5 页，筛选后数据只有 2 页，但页码没变。解决：watch 筛选条件变化时重置 page = 1。

问题三：购物车结算创建多个独立订单。原因：每个选中商品分别调用 orderApi.create，如果中间某个失败，部分订单已创建无法回滚。目前的模拟环境可以接受，但生产环境应该由后端提供批量创建接口。

问题四：骨架屏与实际内容高度不一致导致跳动。解决：骨架屏卡片使用与真实卡片相同的布局结构（图片区 + 文字区），确保高度一致。

问题五：商品图片加载失败破坏布局。解决： 标签的 @error 事件中替换为 Emoji 占位图。

问题六：首页 4 个请求彼此独立，其中一个失败不应影响其他。解决：每个请求都有独立 try-catch，失败后数据为空但页面不崩溃。

12 总结

这个项目中我写的代码主要解决了以下问题：

- **首页如何展示大量商品**：CSS Grid响应式布局 + 骨架屏加载 + 4 个并行 API 请求
- **筛选条件如何持久化**：URL query string同步，刷新页面筛选不丢失
- **购物车的状态管理**：全选/单选用 computed 属性联动，数量修改用防抖优化
- **卖家如何看到并处理自己的订单**：订单列表增加”我买的/我卖的”切换，修复了原始代码写死 buyer 的问题
- **管理后台的数据可视化**：ECharts饼图 + 折线图，带 resize 自适应和 dispose 防泄漏
- **商品数据从哪里来**：155件种子数据 SQL + Python 自定义生成图片，100% 可靠